

Промпт-инжиниринг и работа с кодом

Краткое содержание

Занятие посвящено принципам постановки задач для LLM при работе с кодом.

Ключевая проблема связана с тем, что качество ответа модели зависит от точности промпта, полноты контекста и заданного формата результата.

Материал охватывает базовые техники prompting, способы повышения релевантности ответа, использование RAG и практику работы с AI-агентами в IDE.

Основные понятия

промпт

LLM

контекст

системный промпт

zero-shot

one-shot

few-shot

chain-of-thought

RAG

векторная база данных

embedding

chunk

MCP

AI-агент

Continue

Cline

memory bank

контекстное окно

галлюцинации

релевантность ответа

Основная часть

Промпт как форма инженерной постановки задачи

Промпт в AI-разработке представляет собой структурированный запрос к модели, а не произвольный вопрос. Его задача состоит в том, чтобы передать LLM цель, контекст, ограничения и ожидаемый формат результата. При работе с кодом это особенно критично, поскольку модель должна учитывать язык программирования, версию стека, структуру проекта и принятые соглашения. Нечёткая постановка приводит к нерелевантному коду, лишним зависимостям или решению не той задачи. Поэтому промпт становится частью инженерного workflow и влияет на скорость, стоимость и качество результата.

Структура эффективного промпта

Базовая структура промпта включает роль, контекст, задачу, требования и формат ответа. Роль задаёт профессиональную рамку модели, например backend-разработчик или архитектор. Контекст описывает проект, технологический стек, ограничения и релевантные файлы. Требования фиксируют, что именно должно быть реализовано: обработка ошибок, тесты, документация, стиль кода, формат вывода. Чем точнее заданы входные условия, тем ниже вероятность галлюцинаций и повторной генерации.

Базовые техники prompting

Zero-shot используется, когда задача формулируется без примеров и модель самостоятельно предлагает решение на основе общих знаний. One-shot и few-shot предполагают передачу одного или нескольких примеров, которым модель должна следовать. Эти техники особенно полезны при необходимости сохранить стиль кода, формат endpoint, структуру документации или соглашения об именовании. Chain-of-thought применяется для сложных задач, где требуется пошаговое рассуждение: анализ файла, поиск зависимостей, выявление ошибок, построение плана и только затем генерация кода. Выбор техники зависит от степени неопределённости задачи и требований к контролю результата.

Примеры и пошаговая декомпозиция

Примеры ограничивают пространство решений модели и помогают ей воспроизвести уже принятый в проекте стиль. Если требуется добавить новый endpoint, полезно показать существующий endpoint как образец. Если задача сложная, эффективнее сначала запросить план, а не готовый код. Пошаговая декомпозиция позволяет проверить, как агент понял задачу, и скорректировать ход работы до изменения файлов. Такой подход снижает риск генерации большого объёма некорректного кода.

Оценка качества ответов LLM

Ответ модели требует проверки по релевантности, точности, воспроизводимости, скорости и соответствию формату. Один и тот же промпт может давать разные результаты в разных моделях, поэтому выбор промпта и LLM должен опираться на тестирование. Для прикладных сценариев полезно сравнивать варианты промптов через A/B-подход: менять формулировки, фиксировать результат и выбирать наиболее стабильный вариант. Если система должна работать предсказуемо, промпт должен задавать ограничения, примеры и формат вывода. Без такой проверки LLM остаётся вероятностным генератором, а не управляемым инструментом разработки.

Галлюцинации и обратная связь

Галлюцинации возникают, когда модель достраивает недостающий контекст или уверенно предлагает неверное решение. В коде это проявляется через несуществующие методы, неподходящие библиотеки, неверные контракты API или некорректную обработку ошибок. Для снижения риска требуется передавать актуальные данные проекта, использовать примеры, ограничивать формат ответа и проверять результат тестами. В пользовательских продуктах дополнительно нужна обратная связь от фокус-группы или реальных пользователей. Она позволяет уточнять промпт и корректировать поведение модели.

RAG как расширение контекста

RAG используется, когда модели нужны знания, отсутствующие в её обучающем датасете: новая документация, внутренняя архитектура, корпоративные API или правила проекта. Подход основан на поиске релевантных фрагментов в базе знаний и добавлении их в текущий запрос. Документы разбиваются на chunks, преобразуются в embeddings и сохраняются в векторной базе данных. При запросе система извлекает близкие по смыслу фрагменты и передаёт их модели как дополнительный контекст. RAG не дообучает модель, а временно обогащает её знания для конкретного ответа.

RAG в разработке

В разработке RAG помогает работать с внутренней документацией, релиз-ноутами библиотек, описаниями API, архитектурными решениями и базой знаний команды. Это особенно полезно для больших проектов, где знания распределены между репозиториями, Confluence, спецификациями и внутренними регламентами. Актуальность RAG зависит от регулярного обновления источников и корректной настройки поиска. Если база устарела, модель будет использовать нерелевантные сведения. Поэтому RAG требует не только технической настройки, но и процесса сопровождения знаний.

AI-агенты в IDE

Continue и Cline используются как AI-агенты внутри среды разработки. Они позволяют подключать разные модели, передавать в контекст файлы, папки, документацию, ошибки терминала и элементы кодовой базы. Continue удобен как гибкий open-source-плагин для экспериментов с моделями и настройками. Cline сильнее ориентирован на агентный workflow: планирование, выполнение, контроль контекста, отображение шагов и расхода токенов. Эти инструменты переводят промптинг из режима отдельного чата в режим работы с проектом.

Memory bank и управление контекстом

Memory bank представляет собой сжатое описание проекта, которое агент может использовать как источник контекста. В него включаются стек, структура модулей, ключевые точки входа, архитектурные решения и правила работы с кодом. Такой файл ускоряет онбординг в проект и снижает необходимость каждый раз сканировать всю кодовую базу. При крупных изменениях memory bank должен обновляться, иначе он станет источником устаревших сведений. Для больших проектов целесообразно разделять memory bank по модулям, чтобы не перегружать контекст.

Ограничения контекстного окна

Контекстное окно ограничивает объём данных, который модель может учитывать одновременно. В него попадают запросы, ответы, файлы, документация, планы, ошибки и результаты предыдущих действий. При переполнении контекста часть информации теряется или сжимается, что снижает точность ответа. Поэтому крупные задачи необходимо дробить на отдельные смысловые блоки: анализ backend, анализ

frontend, интеграции, тесты, документация. Такой подход повышает управляемость агента и снижает риск потери важных деталей.

Работа малыми итерациями

Эффективная работа с AI-агентом строится через короткие итерации: постановка задачи, план, выполнение, проверка, уточнение. Если агент движется в неверном направлении, выполнение останавливается, изменения откатываются, а промпт уточняется. Git в этом процессе служит инструментом контроля состояния проекта. Малые изменения проще проверить, протестировать и принять. Чем сложнее задача, тем важнее не поручать агенту весь объем сразу, а управлять последовательностью шагов.

Итоговые выводы

1. Промпт является инженерной постановкой задачи для LLM и должен включать цель, контекст, ограничения и формат результата.
2. Zero-shot, few-shot и chain-of-thought применяются для разных уровней неопределённости и контроля над ответом модели.
3. RAG расширяет контекст модели актуальными документами, но не заменяет дообучение.
4. Continue и Cline позволяют встроить промптинг в рабочий процесс IDE и управлять кодом через AI-агента.
5. Качество AI-разработки зависит от декомпозиции задач, контроля контекста, проверки результата и обновления проектных знаний.